

## ABSTRACT CLASSES

### Aim:

To understand and implement inheritance concepts in Java.

## PRE LAB EXERCISE

### QUESTIONS

- ✓ What is an abstract class?

An abstract class is a class that is declared using the abstract keyword and **cannot be instantiated**. It may contain **abstract methods (without body)** and **concrete methods (with body)**. It is used as a base class for other classes.

- ✓ Why are abstract methods used?

Abstract methods are used to **define a method without implementation**, forcing child classes to provide their own implementation. This ensures **method consistency** and supports **abstraction**.

- ✓ Difference between abstract class and interface.

### Abstract Class

Uses abstract keyword

Can have abstract and non-abstract methods

Can have instance variables

Supports single inheritance

Can have constructors

### Interface

Uses interface keyword

Methods are abstract by default

Variables are public, static, and final

Supports multiple inheritance

Cannot have constructors

## IN LAB EXERCISE

**Objective:**

To implement abstract class and demonstrate abstraction.

**PROGRAMS:****1.University System****Scenario:**

A university has different types of courses: Online, Offline, and Hybrid. Each course has a getDetails() method.

**Question:**

Create an abstract class Course with abstract method getDetails(). Implement OnlineCourse, OfflineCourse, and HybridCourse classes.

**Code:**

```
abstract class Course {  
    abstract void getDetails();  
}
```

```
class OnlineCourse extends Course {  
    void getDetails() {  
        System.out.println("Online Course: Attend via Internet");  
    }  
}
```

```
class OfflineCourse extends Course {  
    void getDetails() {  
        System.out.println("Offline Course: Attend in classroom");  
    }  
}
```

```
class HybridCourse extends Course {  
    void getDetails() {
```

```
        System.out.println("Hybrid Course: Combination of online and offline");
    }
}

public class Main {
    public static void main(String[] args) {
        Course c1 = new OnlineCourse();
        Course c2 = new OfflineCourse();
        Course c3 = new HybridCourse();

        c1.getDetails();
        c2.getDetails();
        c3.getDetails();
    }
}
```

**Output:**

Online Course: Attend via Internet

Offline Course: Attend in classroom

Hybrid Course: Combination of online and offline

```
1 abstract class Course {    Main.java is a non-project file, only syntax errors are reported
2     abstract void getDetails();
3 }
4
5 class OnlineCourse extends Course {
6     void getDetails() {
7         System.out.println(x: "Online Course: Attend via Internet");
8     }
9 }
10
11 class OfflineCourse extends Course {
12     void getDetails() {
13         System.out.println(x: "Offline Course: Attend in classroom");
14     }
15 }
16
17 class HybridCourse extends Course {
18     void getDetails() {
19         System.out.println(x: "Hybrid Course: Combination of online and offline");
20     }
21 }
22
23 public class Main {
24     public static void main(String[] args) {
25         Course c1 = new OnlineCourse();
26         Course c2 = new OfflineCourse();
27     }
28 }
```

Car uses petrol  
Electric car uses battery  
PS C:\Users\ragur> ^C  
PS C:\Users\ragur>  
PS C:\Users\ragur> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '-agentlib:jdwp=transport=dt\_socket,server=n,suspend=y,address=localhost:63945' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ragur\AppData\Local\Temp\vscodesws\_9ab9f\jdt\_ws\jdt.ls-java-project\bin' 'Main'  
Online Course: Attend via Internet  
Offline Course: Attend in classroom  
Hybrid Course: Combination of online and offline  
PS C:\Users\ragur>

## 2. Employee Payroll System

### Scenario:

A company has different types of employees — Regular and Contract. All employees have a salary, but the calculation differs for each type.

### Question:

Design an abstract class Employee with an abstract method calculateSalary(). Implement subclasses RegularEmployee and ContractEmployee to calculate salary differently.

### Code:

```
import java.util.Scanner;

abstract class Employee {

    String name;

    double baseSalary;

    // Abstract method to calculate total salary

    abstract void calculateSalary();
```

```
}
```

```
class RegularEmployee extends Employee {  
    double bonusRate = 0.1; // 10% bonus  
  
    void calculateSalary() {  
        double totalSalary = baseSalary + (baseSalary * bonusRate);  
        System.out.println("Regular Employee: " + name);  
        System.out.println("Base Salary: " + baseSalary);  
        System.out.println("Total Salary (with 10% bonus): " + totalSalary);  
    }  
}
```

```
class ContractEmployee extends Employee {  
    void calculateSalary() {  
        System.out.println("Contract Employee: " + name);  
        System.out.println("Total Salary: " + baseSalary);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        // Input for Regular Employee  
        System.out.print("Enter Regular Employee Name: ");  
        String regName = sc.nextLine();  
        System.out.print("Enter Base Salary: ");
```

```
double regSalary = sc.nextDouble();
sc.nextLine(); // Consume newline

// Input for Contract Employee
System.out.print("Enter Contract Employee Name: ");
String conName = sc.nextLine();
System.out.print("Enter Base Salary: ");
double conSalary = sc.nextDouble();

// Create objects
Employee e1 = new RegularEmployee();
e1.name = regName;
e1.baseSalary = regSalary;

Employee e2 = new ContractEmployee();
e2.name = conName;
e2.baseSalary = conSalary;

System.out.println("\n--- Salary Details ---");
e1.calculateSalary();
System.out.println();
e2.calculateSalary();

sc.close();
}
}
```

**Output:**

Enter Regular Employee Name: Ram

Enter Base Salary: 30000

Enter Contract Employee Name: Ravi

Enter Base Salary: 20000

--- Salary Details ---

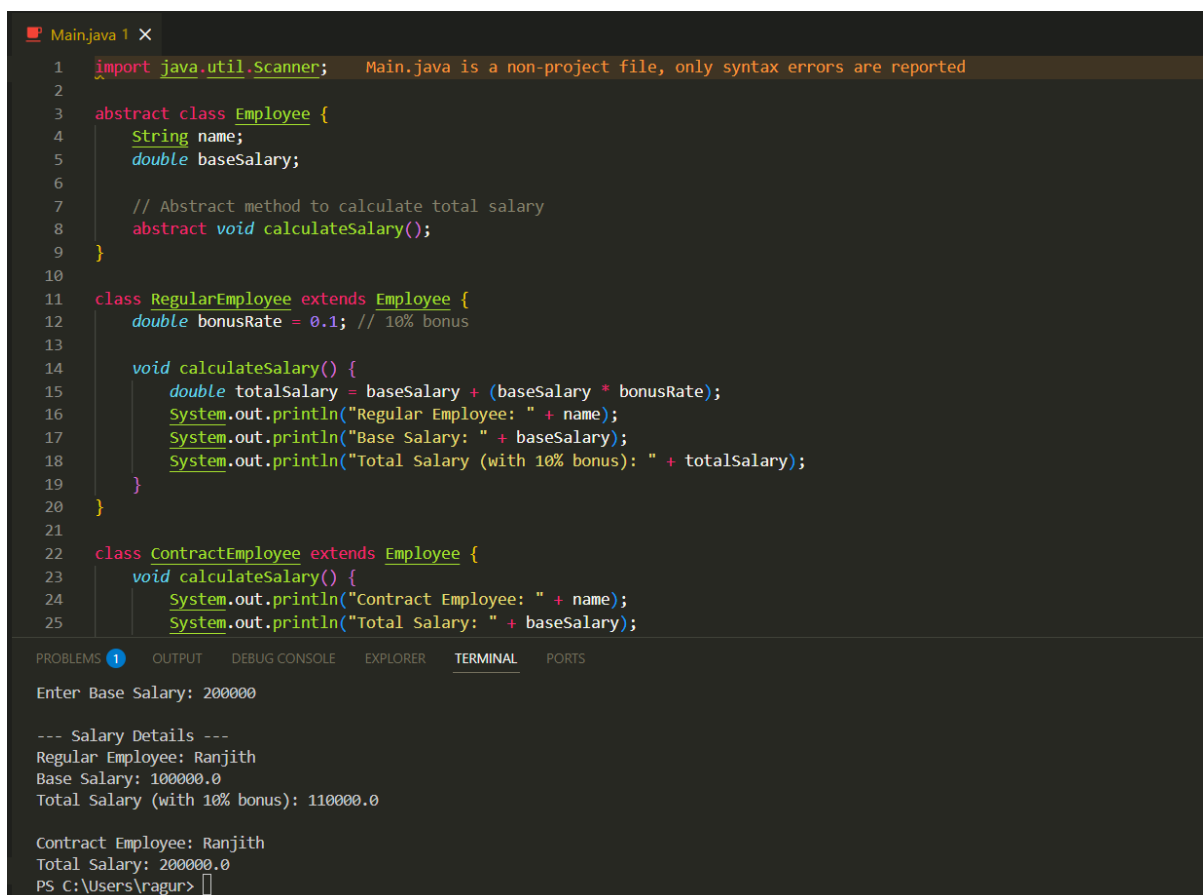
Regular Employee: Anitha

Base Salary: 30000.0

Total Salary (with 10% bonus): 33000.0

Contract Employee: Ravi

Total Salary: 20000.0



```
1  import java.util.Scanner;    Main.java is a non-project file, only syntax errors are reported
2
3  abstract class Employee {
4      String name;
5      double baseSalary;
6
7      // Abstract method to calculate total salary
8      abstract void calculateSalary();
9  }
10
11  class RegularEmployee extends Employee {
12      double bonusRate = 0.1; // 10% bonus
13
14      void calculateSalary() {
15          double totalSalary = baseSalary + (baseSalary * bonusRate);
16          System.out.println("Regular Employee: " + name);
17          System.out.println("Base Salary: " + baseSalary);
18          System.out.println("Total Salary (with 10% bonus): " + totalSalary);
19      }
20  }
21
22  class ContractEmployee extends Employee {
23      void calculateSalary() {
24          System.out.println("Contract Employee: " + name);
25          System.out.println("Total Salary: " + baseSalary);
26      }
27  }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE EXPLORER TERMINAL PORTS

Enter Base Salary: 200000

--- Salary Details ---

Regular Employee: Ranjith  
Base Salary: 100000.0  
Total Salary (with 10% bonus): 110000.0

Contract Employee: Ranjith  
Total Salary: 200000.0

PS C:\Users\ragur> |

### 3.Banking System

Scenario:

A bank has different types of accounts: Savings and Current. Both accounts need a method to calculate interest, but the calculation differs for each account type.

**Question:**

Use an abstract class BankAccount with an abstract method calculateInterest() and implement it in SavingsAccount and CurrentAccount classes.

**Code**

```
abstract class BankAccount {  
    String accountHolder;  
    double balance;  
  
    BankAccount(String name, double bal) {  
        accountHolder = name;  
        balance = bal;  
    }  
  
    abstract void calculateInterest(); // Abstract method  
}  
  
class SavingsAccount extends BankAccount {  
    double interestRate = 0.04; // 4% interest  
  
    SavingsAccount(String name, double bal) {  
        super(name, bal);  
    }  
  
    void calculateInterest() {  
        double interest = balance * interestRate;  
        System.out.println("Savings Account Interest for " + accountHolder + " = " +  
interest);  
    }  
}
```



```

    }
}

class CurrentAccount extends BankAccount {
    double interestRate = 0.02; // 2% interest

    CurrentAccount(String name, double bal) {
        super(name, bal);
    }

    void calculateInterest() {
        double interest = balance * interestRate;

        System.out.println("Current Account Interest for " + accountHolder + " = " +
interest);
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc1 = new SavingsAccount("Ram", 50000);
        BankAccount acc2 = new CurrentAccount("Ravi", 80000);

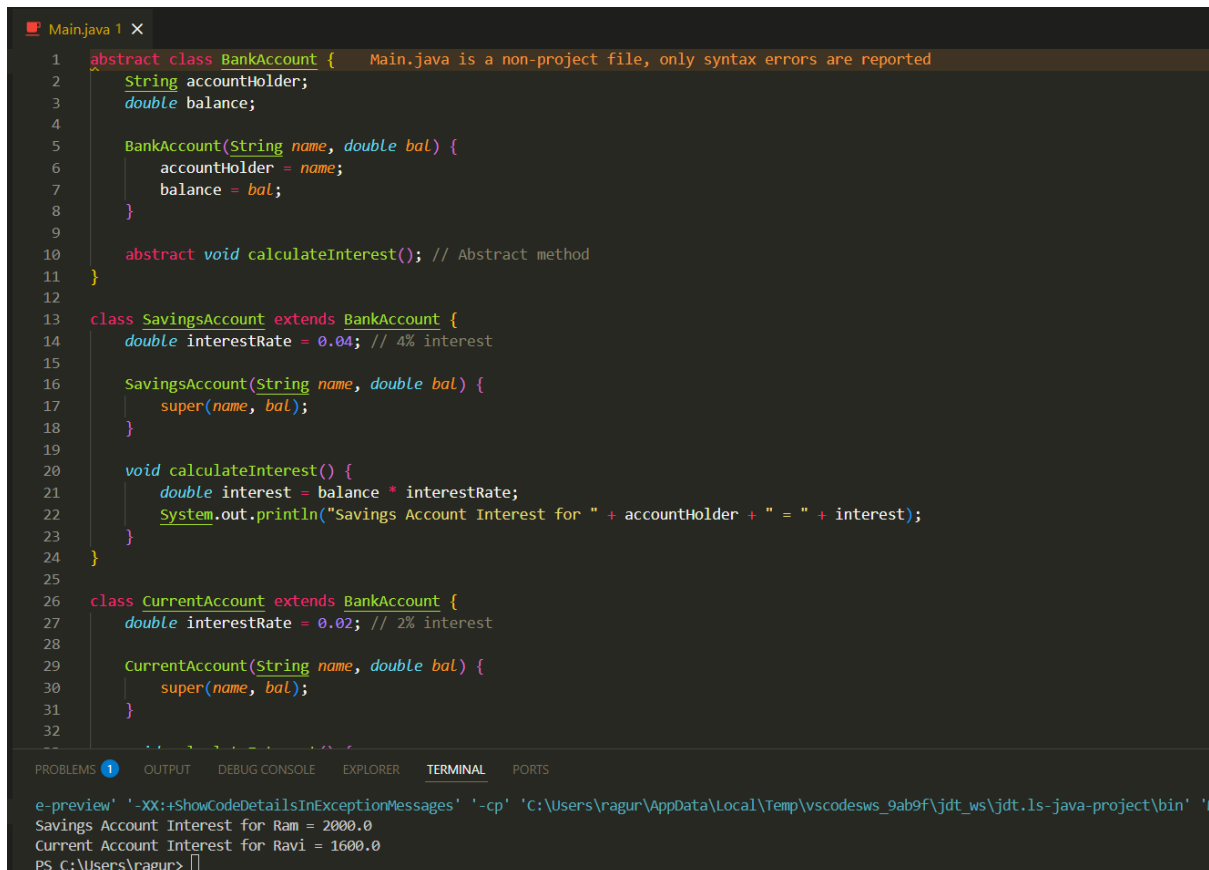
        acc1.calculateInterest();
        acc2.calculateInterest();
    }
}

```

### **Output**

Savings Account Interest for Ram = 2000.0

Current Account Interest for Ravi = 1600.0



```
1  abstract class BankAccount {    Main.java is a non-project file, only syntax errors are reported
2      String accountHolder;
3      double balance;
4
5      BankAccount(String name, double bal) {
6          accountHolder = name;
7          balance = bal;
8      }
9
10     abstract void calculateInterest(); // Abstract method
11 }
12
13 class SavingsAccount extends BankAccount {
14     double interestRate = 0.04; // 4% interest
15
16     SavingsAccount(String name, double bal) {
17         super(name, bal);
18     }
19
20     void calculateInterest() {
21         double interest = balance * interestRate;
22         System.out.println("Savings Account Interest for " + accountHolder + " = " + interest);
23     }
24 }
25
26 class CurrentAccount extends BankAccount {
27     double interestRate = 0.02; // 2% interest
28
29     CurrentAccount(String name, double bal) {
30         super(name, bal);
31     }
32 }
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE EXPLORER TERMINAL PORTS

```
e-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ragur\AppData\Local\Temp\vscode\9ab9f\jdt_ws\jdt.ls-java-project\bin' 'h
Savings Account Interest for Ram = 2000.0
Current Account Interest for Ravi = 1600.0
PS C:\Users\ragur>
```

## POST LAB EXERCISE

### ✓ Difference between abstract and regular class:

An abstract class cannot be instantiated and may have abstract methods. A regular class can be instantiated and has only implemented methods.

### ✓ Can we create an object of an abstract class?

No. Because it may contain unimplemented (abstract) methods.

### ✓ If subclass does not implement abstract methods?

The subclass must also be declared abstract. Otherwise, a compilation error occurs.

### ✓ Can abstract class have no abstract methods?

Yes. It can prevent object creation and act as a base class.

### ✓ Can abstract class extend another abstract class?

Yes. It can implement methods or leave them for its subclasses.

**Result:**

Thus the abstract classes and methods were implemented and executed successfully.

**ASSESSMENT**

| Description              | Max Marks | Marks Awarded |
|--------------------------|-----------|---------------|
| Pre Lab Exercise         | 5         |               |
| In Lab Exercise          | 10        |               |
| Post Lab Exercise        | 5         |               |
| Viva                     | 10        |               |
| <b>Total</b>             | <b>30</b> |               |
| <b>Faculty Signature</b> |           |               |